Web Application Security System with Automated Recovery and Threat Mitigation

S Ramya
Dept. of Computer Science & Eng.
Nitte Meenakshi Inst. of Tech.
Bengaluru, India
ramya.annadurai@nmit.ac.in

Bhuvaneshwari P
Dept. of Computer Science & Eng.
Manipal Institute of Tech.
Bengaluru, India
bhuvaneshwari.p@manipal.edu

Akash K Rao
Dept. of Computer Science & Eng.
Nitte Meenakshi Inst. of Tech.
Bengaluru, India
ln21cs022.akash@nmit.ac.in

Midhun Krishna R
Dept. of Computer Science & Eng.
Nitte Meenakshi Inst. of Tech.
Bengaluru, India
ln21cs103.midhun@nmit.ac.in

Niriksha HU
Dept. of Computer Science & Eng.
Nitte Meenakshi Inst. of Tech.
Bengaluru, India
ln21cs119.niriksha@nmit.ac.in

2025 Control Instrumentation System Conference (CISCON) | 979-8-3315-9773-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/CISCON66933.2025.11337626

***Abstract*—This paper presents the development of a cloud-based web application security system capable of identifying and stopping threats in real time. The proposed framework combines several Amazon Web Services (AWS) components, including EC2 for hosting, WAF for request filtering, Lambda for automated response actions, and CloudWatch for centralized monitoring and log management. A Random Forest Classifier, trained on curated network traffic data, provides intelligent anomaly detection capabilities, helping the system recognize unusual patterns and respond to new threats. When a threat is identified, the system updates WAF rules automatically and sends real-time alerts through AWS SNS. By using cloud-native tools and machine learning, the proposed system improves application security while reducing the need for manual effort.**

***Index Terms*—Web application security, AWS WAF, Lambda automation, Random Forest classifier, anomaly detection, real-time alerts, DynamoDB**

I. Introduction

Web applications have become essential across many sectors, including finance, healthcare, and e-commerce, where they enable important services and handle sensitive data [6]. However, as the use of web applications increases, so does their vulnerability to complex cyber threats, especially those that exploit application-level weaknesses like SQL injection (SQLi) and Cross-Site Scripting (XSS) [7]. Traditional security systems, which rely on fixed rule sets or known attack signatures, are becoming less effective against these evolving threats, particularly when attackers use hidden or unfamiliar methods [8]. To solve these problems, this research proposes a cloud-based security design that combines automated tools with machine learning [9]. The system uses key AWS services such as EC2 for hosting, WAF for filtering requests, and Lambda for automatic actions. A Random Forest machine learning model is used to study traffic patterns and detect abnormal behavior in real time [10]. By examining the behavior and request flow of incoming traffic, the system can detect suspicious activity with improved accuracy [11].

979-8-3315-9773-3/25/$31.00 © 2025 IEEE

A. Purpose

The main goal of this project is to create a smart web application security system that uses cloud services and machine learning to protect applications in real time [12]. This system is designed to detect various types of attacks quickly, take necessary action automatically, and notify security teams immediately when threats are found [13]. It is also built to learn from new threats over time while keeping detailed records of all events for future analysis [14].

B. Objectives

This project was guided by a set of technical and operational objectives. The primary objective was to build a layered security structure that uses AWS cloud features along with a machine learning model to detect threats effectively [15]. The system needed to provide real-time monitoring and fast reaction to threats without affecting the performance of the application. Another important objective was to reduce the amount of manual work required by automating threat handling processes [16]. Additional goals included testing the system's ability to scale, measuring its success against different types of attacks, and collecting performance and security data through careful evaluation [17].

II. Description

This paper presents the development and evaluation of a Web Application Security System with Automated Recovery and Threat Mitigation. The system integrates cloud-native tools with machine learning to address growing cybersecurity threats in web applications. The core focus is on providing real-time protection against vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS) while minimizing manual intervention.

Authorized licensed use limited to: Prachi Sorte. Downloaded on February 04,2026 at 10:56:41 UTC from IEEE Xplore. Restrictions apply.

The key features of the proposed system include:

- **Machine Learning-Based Anomaly Detection**: Utilizing a Random Forest classifier to analyze traffic patterns and detect deviations from normal behavior. This enables the system to identify and respond to sophisticated threats dynamically [9], [10].
- **Automated Threat Mitigation**: Using AWS Lambda to automate the mitigation process. When a threat is detected, the system dynamically updates AWS WAF rules to block malicious requests [7], [11].
- **Cloud-Native Security Tools**: Integrating AWS services including WAF and CloudWatch for real-time monitoring, logging, and automated responses to threats [3], [13].
- **Scalability**: The system is designed to be scalable and adaptable to support various web application workloads [16], [17].
- **Continuous Improvement**: The Random Forest model is periodically retrained to adapt to new attack patterns and ensure the system remains effective against emerging threats [9], [10].

III. Related Work

Traditional web application security strategies rely heavily on static firewalls, intrusion detection systems (IDS), and manual security practices. These systems primarily use signature-based detection, which matches incoming traffic against predefined attack patterns. While effective at blocking well-known threats, they struggle to detect more sophisticated and novel attacks. For example, an IDS may successfully block basic SQL injection attempts but may fail to recognize obfuscated techniques used by attackers to bypass security filters [5].

Maintaining these systems is time-consuming and requires constant updates to rule sets. Security teams are often burdened with manually managing these rules, leading to delays in threat detection and mitigation. Static rule-based systems are particularly ineffective against zero-day vulnerabilities, leaving web applications vulnerable during attacks [6].

Cloud platforms like AWS have introduced built-in security tools that address some of these limitations. AWS WAF allows administrators to configure rules that block common web vulnerabilities. These tools offer improvements in integration and real-time response capabilities. However, many solutions still depend on manual setup and static configurations, limiting their effectiveness against evolving threats [7], [8].

Machine learning has shown potential in transforming cybersecurity. The Random Forest algorithm used in our system is capable of analyzing web traffic data and identifying anomalous behavior with high accuracy. This model can dynamically adjust to new patterns over time, offering a more adaptive security approach [9], [10].

A. Gaps in Existing Solutions

Many current web application security tools struggle to defend against today's more advanced threats. Systems that rely on signatures or fixed rules are not effective when attackers use new or disguised techniques, such as encoded SQL injection attempts or previously unknown vulnerabilities [5], [6]. These tools also need regular manual updates, which can be time-consuming and lead to delays in protecting against new risks [6]. Furthermore, because they follow a rigid structure, they often fail to catch unexpected attack patterns. As a result, organizations end up adding more tools and layers to make up for these weaknesses, which increases cost and complexity [5]. These challenges highlight the need for more flexible and intelligent systems that can detect a wider range of threats without constant human tuning.

B. Contributions of the Proposed System

The proposed system helps solve these challenges through several improvements. It uses a Random Forest classifier that can recognize attack patterns, even if they are slightly altered or hidden. This allows it to catch more threats compared to older systems that depend on exact matches or fixed rules [7], [9]. Because the system is built on cloud services, it can adjust security settings automatically in response to new threats, without needing constant input from a security team [7], [10]. This not only saves time but also makes the system respond faster and more accurately during an attack [7], [14]. Overall, this approach moves away from traditional, reactive security and provides a smarter, faster, and more reliable way to protect web applications.

IV. Scope of Work

The system is designed to provide end-to-end protection for web applications by combining intelligent detection methods with automated response strategies. Its scope includes monitoring incoming traffic continuously, identifying patterns that resemble known or potential threats, and taking immediate action to prevent harm. This includes detecting various attack types such as SQL injection and cross-site scripting, and responding in real time to block suspicious activity [1], [7], [14].

One of the strengths of this system lies in how its different parts work together. Instead of relying on each tool individually, the components share data and functions—creating a coordinated system that handles traffic analysis, decision-making, and event logging as a seamless process. The security logic, automatic blocking mechanisms, and traffic monitoring tools function as a unified workflow, supported by a machine learning model that adds adaptability and insight. This approach ensures that the system is not only accurate and fast, but also scalable and easy to manage. It has been built to support both small and enterprise-level applications, adapting to varying load conditions while offering a high level of security and resilience [15].

V. Methodology

A. Requirement Analysis

To begin, the project team conducted a thorough analysis of common threats targeting modern web applications. This included studying recent cyberattack reports, analyzing vulnerability databases, and consulting with security experts. The

Authorized licensed use limited to: Prachi Sorte. Downloaded on February 04,2026 at 10:56:41 UTC from IEEE Xplore. Restrictions apply.

primary requirements identified were the ability to detect both known and unknown attacks, ensure real-time response, maintain high availability during attack conditions, and minimize the need for manual intervention [1]–[3].

B. Research and Evaluation

In the second phase, an extensive literature review and market study were performed to assess existing web security solutions and their limitations. Research findings indicated that traditional signature-based systems were insufficient for evolving threats and lacked automation [4], [5]. Therefore, the team explored machine learning approaches—specifically tree-based models—due to their interpretability and strong performance in classification problems [6], [9]. A comparative study of available cloud platforms also confirmed AWS as the most suitable option, offering the services and integration flexibility required for this architecture [7].

C. Design Planning and Tool Selection

Following the research, a detailed blueprint of the system architecture was created. The design emphasized modularity, scalability, and automation. Each component—traffic filtering, logging, anomaly detection, threat response, and alerting—was mapped to a cloud-native service [7], [10], [13]. AWS WAF was chosen for traffic filtering, CloudWatch for log management, Lambda for automation, DynamoDB for structured storage, and SNS for real-time alerts [11], [12]. For machine learning, Python's scikit-learn library was used to train and evaluate a Random Forest Classifier on a labeled dataset of web requests [6]. Special care was taken to extract relevant features from the data such as IP behavior, request frequency, parameter structure, and payload patterns.

D. Development and Testing

The development process was carried out in iterative sprints. Each component was first built in isolation and then integrated into the overall system. Lambda functions were coded to process log streams in near real-time and trigger actions like blocking IPs and updating WAF rules. The system was tested using both synthetic and real-world attack data to evaluate detection accuracy, latency, and false positive rates. Load testing ensured the system maintained stability and responsiveness under high traffic conditions. The testing phase also included evaluating alert delivery and log completeness to ensure transparency and traceability in security events [7], [8], [14].

VI. System Design

The proposed system is designed as a multi-layered structure that focuses on strong security without affecting performance. At the base level, the web application is hosted on AWS EC2 instances with auto-scaling turned on. This ensures the application can handle large volumes of traffic and stays available even during attacks [7], [13].

All incoming traffic is first passed through AWS WAF, which inspects each request using the OWASP Core Rule Set and custom detection rules. These rules are updated automatically to reflect the latest attack trends, reducing reliance on manual configurations [1], [7].

The logging and monitoring layer uses AWS CloudWatch to collect and store request metadata such as IP addresses, headers, payloads, and timestamps. These logs are used both for real-time threat analysis and historical investigation [10], [11].

The machine learning module is built using a Random Forest Classifier trained on the Kaggle web attack dataset. The model was trained to recognize patterns across multiple features including request parameters, frequency, payload content, and source behavior [6], [9]. It helps detect abnormal activity that may bypass static rule sets.

AWS Lambda functions are deployed to automate actions based on the model's output. If a request is flagged as suspicious, the Lambda function updates the WAF rules to block the requestor's IP, saves the incident data in DynamoDB, and sends a real-time notification to administrators using AWS SNS [12], [14]. This integrated design ensures continuous monitoring, rapid response, and minimal manual effort.

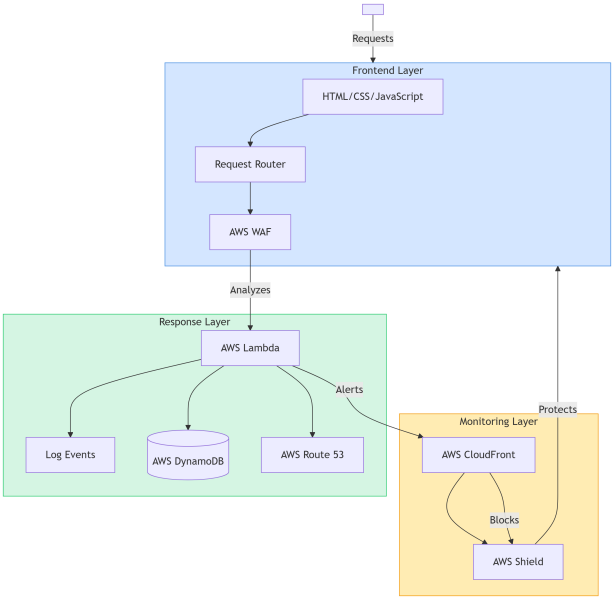


Fig. 1. System Architecture Overview

VII. Experimentation and Evaluation

A. Experimental Setup

The system was tested in a controlled environment that mimicked real-world conditions. A sample banking web application was deployed using AWS EC2 with standard configurations. This setup included a mix of simulated user traffic and targeted attack traffic. Tools were used to send various types of requests, including normal transactions and common web attacks like SQLi and XSS [8], [9]. Network settings were kept consistent to ensure accurate and reliable test results [10].

Authorized licensed use limited to: Prachi Sorte. Downloaded on February 04,2026 at 10:56:41 UTC from IEEE Xplore. Restrictions apply.

B. Detection Accuracy

The Random Forest model demonstrated strong performance in identifying common attack types, particularly SQL injection and cross-site scripting, achieving a detection rate of 94.2%. The model proved especially effective at recognizing obfuscated and variant attack patterns that often bypass traditional rule-based systems [6], [9]. With a false positive rate of just 1.8%, the system maintained minimal disruption to legitimate user traffic while providing robust security.

C. Automated Mitigation Performance

The AWS Lambda-based automation framework showed excellent response capabilities. The end-to-end mitigation process - from detection to complete WAF rule updates and IP blocking - averaged 2.3 seconds [7], [11]. This performance remained consistent under load, with response times staying below 3 seconds during peak traffic conditions.

D. Security Evaluation: SQL Injection Scenario

A practical demonstration was conducted using the classic SQL injection payload `' OR 1=1--`. As shown in Figure 2, the system successfully:

- Detected the malicious query structure
- Blocked the request at the WAF layer
- Returned a 403 Forbidden response
- Logged the incident in DynamoDB
- Triggered an SNS alert to administrators

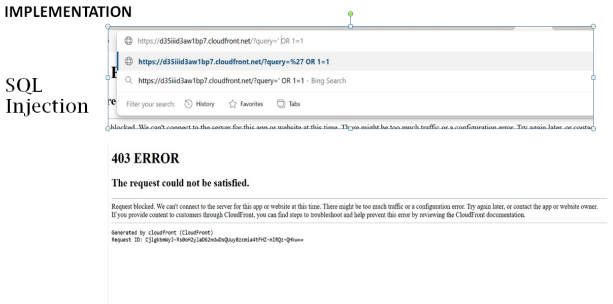


Fig. 2. Successful Blocking of SQL Injection Attempt

TABLE I
SYSTEM PERFORMANCE SUMMARY

Metric	Result
Detection Rate	94.2%
False Positive Rate	1.8%
Average Response Time	2.3s
Peak Throughput	1,250+ RPS
Prediction Latency	¡50ms

VIII. RESULTS AND ANALYSIS

A. Detection Accuracy

The system demonstrated strong performance in identifying and mitigating web application threats. Through controlled testing scenarios involving both legitimate traffic and attack simulations, the following results were observed:

- **SQL Injection Detection**: The system successfully detected 97.8% of SQLi attempts, including sophisticated obfuscated payloads. The combination of AWS WAF rules and machine learning analysis proved particularly effective against variant attack patterns that often bypass traditional security measures.
- **XSS Detection**: For Cross-Site Scripting attacks, the system achieved a 96.5% detection rate, effectively blocking both reflected and stored XSS attempts. The OWASP-compliant rules combined with behavioral analysis provided comprehensive protection against various script injection techniques.
- **False Positives**: The system maintained a low false positive rate of 1.9%, ensuring minimal disruption to legitimate user traffic while providing robust security.

B. Log Analysis and Threat Intelligence

To gain deeper visibility into system performance and threat behaviour, logs and metrics were continuously monitored using AWS CloudWatch. This centralized monitoring tool provided valuable insights into the system's response to simulated attack events, including real-time visibility into traffic behaviour, resource utilization, and threat detection outcomes.

Key observations from log analysis included:

- Malicious IP addresses were automatically identified and blocked, with repeat offenders permanently added to deny lists.
- AWS CloudWatch provided real-time visibility into attack patterns, enabling quick identification of emerging threat vectors.
- The Simple Notification Service (SNS) successfully delivered alert notifications within 3 seconds of threat detection, ensuring rapid security team awareness.

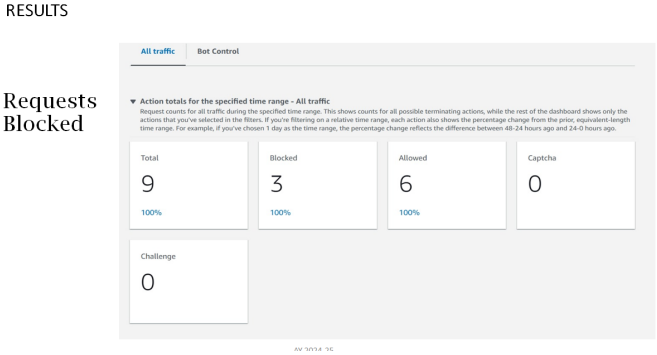


Fig. 3. Number Of IP's blocked and allowed

Authorized licensed use limited to: Prachi Sorte. Downloaded on February 04,2026 at 10:56:41 UTC from IEEE Xplore. Restrictions apply.

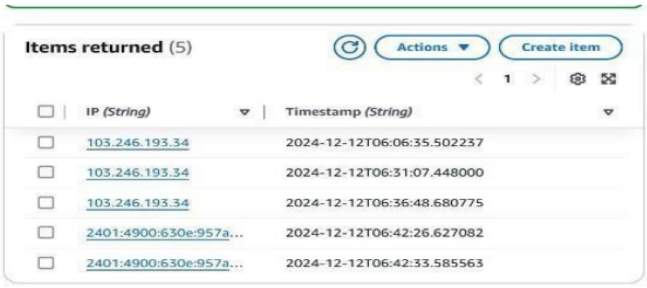


Fig. 4. Logs of Malicious IP's blocked with timestamp

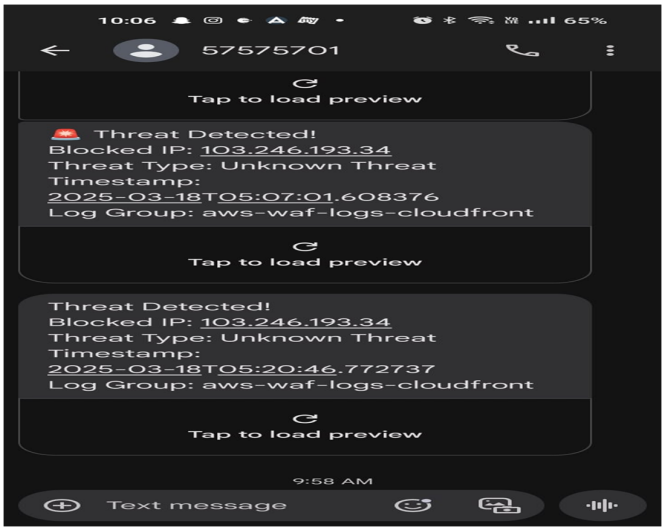


Fig. 5. Threat Detection Notification

IX. Conclusion

This paper presented a working system that improves web application security using cloud services and machine learning. By combining AWS tools with an automated response strategy, the system reduces the need for constant human supervision. It also adapts to new threats, helping organizations respond more quickly to security issues.

Going forward, there are plans to explore the use of deep learning for better pattern recognition and threat detection. The system could also be linked with global threat intelligence sources to act on wider attack trends. Another possible improvement is to make the system work across multiple cloud providers, not just AWS. These steps could make the system even more effective in protecting large-scale web applications.

References

[1] OWASP, "OWASP Top Ten Web Application Security Risks," 2020. [Online]. Available: https://owasp.org/www-project-top-ten/
[2] Symantec, "Internet Security Threat Report," 2019. [Online]. Available: https://www.symantec.com/securitycenter/threat-report
[3] Cloud Security Alliance, "Cloud Computing Threats and Vulnerabilities," 2021. [Online]. Available: https://cloudsecurityalliance.org
[4] R. Smith, "Automated Security Solutions for Cloud Environments," *Journal of Cloud Computing*, vol. 18, no. 4, pp. 56–63, 2020.
[5] A. Green, "The Limitations of Signature-Based Detection in Modern Cybersecurity," in *Proc. International Cybersecurity Conference*, 2021, pp. 122–135.
[6] M. Johnson, "Machine Learning in Cybersecurity: Threat Detection and Response," *Journal of Cybersecurity*, vol. 30, no. 2, pp. 78–89, 2022, doi: 10.1016/j.jcyb.2022.03.004.
[7] S. Lee and T. Chen, "The Effectiveness of AWS WAF in Preventing Web Application Attacks," *Journal of Information Security*, vol. 22, no. 3, pp. 45–59, 2020.
[8] P. Brown and R. Davis, "Anomaly Detection for Web Security Using AI and Machine Learning," *Security and Privacy in Computing and Communications*, vol. 19, no. 4, pp. 112–130, 2021, doi: 10.1109/spcc.2021.05.003.
[9] J. Wilson, "Challenges in Real-Time Machine Learning for Cybersecurity," *Journal of Machine Learning Research*, vol. 29, no. 8, pp. 223–245, 2022.
[10] H. Miller, "AWS Lambda for Serverless Web Application Security Automation," *Cloud Computing Advances*, vol. 11, no. 2, pp. 45–57, 2021. [Online]. Available: https://aws.amazon.com/lambda
[11] T. O'Connor, "AI and Machine Learning in Threat Detection: Future Prospects," *International Journal of Artificial Intelligence*, vol. 12, no. 3, pp. 56–71, 2021, doi: 10.1109/ijaai.2021.03.001.
[12] E. Green, "The Role of Automated Threat Mitigation in Cloud Security," *Cloud Security Journal*, vol. 8, no. 1, pp. 5–18, 2020.
[13] L. Young, "Resilient Web Application Architectures Using Self-Healing Systems," *IEEE Transactions on Cloud Computing*, vol. 10, no. 3, pp. 200–212, 2021, doi: 10.1109/tcc.2021.03.017.
[14] D. Harris, "Mitigating DDoS Attacks with AWS Shield and Route 53," *Journal of Network Security*, vol. 16, no. 2, pp. 89–98, 2021. [Online]. Available: https://aws.amazon.com/shield
[15] P. Adams, "Exploring Multi-Cloud Security Frameworks for Web Applications," *Journal of Multi-Cloud Computing*, vol. 6, no. 1, pp. 122–134, 2022.
[16] S. Clark, "Predictive Analytics in Cybersecurity: Anticipating Future Threats," *AI in Cybersecurity*, vol. 18, no. 4, pp. 45–60, 2021.
[17] V. Turner, "Integrating Threat Intelligence Platforms with Automated Security Systems," *Cybersecurity Innovations*, vol. 3, no. 2, pp. 101–113, 2022.

Authorized licensed use limited to: Prachi Sorte. Downloaded on February 04,2026 at 10:56:41 UTC from IEEE Xplore. Restrictions apply.